

Unit 5

React js

Exam-Ready Study Notes

React – History, Evolution & Why It Was Created (Very Important)

■ Definition

- React is a JavaScript library for building user interfaces, developed by Facebook, which focuses on creating fast, interactive, and reusable UI components using a component-based architecture.

■ Introduction

Before React, web applications were built using traditional JavaScript or frameworks like jQuery, where developers manually updated the DOM. As applications became complex (especially social media apps like Facebook), managing UI updates became difficult, slow, and error-prone.

React was introduced to solve these problems by providing a declarative and efficient way to build dynamic user interfaces.

■ React History

React was developed by Jordan Walke, a software engineer at Facebook.

Timeline:

- 2011 → React was first used internally at Facebook
- 2012 → Used in Instagram
- 2013 → React was officially released as open source
- 2015+ → Became widely popular and adopted by companies like Netflix, Airbnb
- 2019+ → Introduction of Hooks (modern React)
- Present → One of the most widely used frontend libraries

React is maintained by Facebook (Meta) and a large open-source community.

■ Facebook Messaging State Synchronization Problem (Core Reason for React)

■ Problem Explanation

Facebook faced a major issue in its messaging/chat system:

- Messages were updating in multiple places:
 - Chat window
 - Notifications
 - Sidebar
- When one component updated, others didn't update correctly
- This led to:
 - Inconsistent UI
 - Out-of-sync data
 - Complex code to manually manage updates

■ This problem is called State Synchronization Problem

■ Real-World Example

Imagine:

- You send a message
- It appears in:
 - Chat window
 - Notification count (not updated)
 - Recent chats

This creates inconsistent UI state.

■ Solution by React

React introduced:

1. Single Source of Truth (State)
2. Component-based structure
3. Automatic UI updates when state changes

■ When state changes → React automatically updates all related UI parts

This solved:

- Synchronization issues
- Manual DOM manipulation
- Complex UI logic

■ Evolution of React

React has evolved significantly over time:

1. Initial Version (2013)
 - Class-based components
 - Manual lifecycle methods
 - Complex syntax
2. Introduction of Virtual DOM
 - Improved performance
 - Faster updates
 - Efficient rendering
3. ES6 & Functional Components
 - Simpler syntax
 - More readable code
4. React Hooks (2019 – Major Update)
 - Allowed state in functional components
 - Replaced many lifecycle methods
 - Made code cleaner and reusable

Example:

```
import React, { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

5. Modern React

- Hooks-based development
- Functional components preferred
- Better performance and scalability

■ Key Concepts Introduced by React

- Component-Based Architecture
- Virtual DOM
- One-Way Data Binding
- Reusable UI Components
- Declarative Programming

■ Advantages of React

- Fast rendering using Virtual DOM
- Reusable components
- Easy to maintain

* Strong community support

- Suitable for large-scale applications

■ Disadvantages of React

- Learning curve for beginners
- Requires additional tools (routing, state management)
- Frequent updates and changes

■ Conclusion

React was developed to solve real-world UI problems faced by Facebook, especially state synchronization issues in complex applications. Over time, it evolved into a powerful and flexible library that simplifies frontend development using components and efficient rendering techniques.

Today, React is one of the most important technologies in full stack development.

Need of React (Why React is Required)

■ Definition

The Need of React refers to the reasons why React was developed and why it is used in modern web development to build fast, scalable, and interactive user interfaces efficiently.

■ Introduction

In earlier web development, developers used HTML, CSS, and JavaScript (or jQuery) to build websites. While this approach worked for small applications, it became difficult to manage large and dynamic applications like Facebook, Instagram, or Gmail.

As applications grew in complexity, developers faced issues such as:

- Slow performance
- Difficult DOM manipulation
- Code duplication
- Poor maintainability

React was introduced to solve these problems by providing a structured, efficient, and component-based approach.

■ Problems in Traditional Web Development

1. Manual DOM Manipulation

In traditional JavaScript:

- Developers manually update the DOM using methods like:

```
document.getElementById("text").innerHTML = "Hello";
```

- This becomes complex and error-prone for large applications

■ Problem:

- Hard to track UI changes
- Leads to bugs and inconsistencies

2. Poor Performance

- Every DOM update directly affects the real DOM
- Real DOM operations are slow and expensive

■ Example:

- Updating a small part may re-render the entire page

3. Code Reusability Issues

- No proper structure for reuse
- Developers write repeated code

■ Problem:

- Difficult to maintain
- Increases development time

4. State Management Complexity

- Data changes in one part of the app were not reflected in others
- Developers had to manually sync data

■ Leads to:

- State synchronization problems (like Facebook messaging issue)
5. Difficult to Build Large Applications
- Code becomes messy
 - Hard to debug
 - Not scalable

■ How React Solves These Problems

React introduces powerful concepts to overcome these limitations:

1. Virtual DOM (Improves Performance)
- React uses a Virtual DOM (lightweight copy of real DOM)
 - It updates only the changed parts instead of the entire DOM

■ Benefit:

- Faster updates
 - Better performance
2. Component-Based Architecture
- UI is divided into small reusable components

■ Example:

- Navbar component
- Footer component
- Button component

■ Benefit:

- Code reusability
 - Easy maintenance
3. Declarative Approach
- Developers describe what UI should look like, not how to update it

■ Example:

```
function App() {  
  return <h1>Hello World</h1>;  
}
```

■ Benefit:

- Simpler and cleaner code
4. One-Way Data Binding
- Data flows in one direction (Parent → Child)

■ Benefit:

- Predictable behavior
- Easier debugging

5. Efficient State Management

- React automatically updates UI when state changes

■ Example:

```
import React, { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

■ Benefit:

- No manual synchronization needed
- ### 6. JSX (JavaScript + HTML)
- Allows writing HTML inside JavaScript

■ Benefit:

- Improves readability
- Reduces code complexity

■ Real-World Example

Consider a shopping website:

Without React:

- Updating cart requires manual DOM updates
- High chance of bugs

With React:

- Cart is a component
- When item is added → UI updates automatically

■ This makes development faster and more reliable

■ Advantages of Using React

- High performance due to Virtual DOM
- Reusable components
- Easy to debug
- Scalable for large applications

* Strong community support

■ Disadvantages of React

- Requires learning JSX
- Needs additional tools (Redux, Router)
- Frequent updates can be confusing

■ Conclusion

React is needed to overcome the limitations of traditional web development by providing a fast, efficient, and scalable way to build modern user interfaces. It simplifies UI development using components, virtual DOM, and one-way data flow, making it ideal for large and dynamic applications.

TIP: Exam Writing Tips

- Start with problems in traditional DOM (very important)
- Mention:
 - Virtual DOM
 - Component-based architecture
 - One-way data flow
- Include one small code example
- Use keywords like:
 - Performance*
 - Reusable components*
 - Efficient UI updates*

Simple React Structure (React Application Structure & File Explanation)

■ Definition

A Simple React Structure refers to the basic folder and file organization of a React application, which helps developers build applications using a component-based architecture in a clean and maintainable way.

■ Introduction

React applications are not written in a single file. Instead, they are organized into multiple folders and files, where each file represents a specific part of the application.

This structured approach helps:

- Divide application into small components
- Improve code readability
- Make development scalable

■ Most React applications are created using tools like Create React App or Vite, which automatically generate this structure.

■ Basic React Project Structure

A typical React project looks like this:


```
my-app/  
├──  
│   ├── node_modules/  
│   ├── public/  
│   │   └── index.html  
│   └──  
│       ├── src/  
│       │   ├── App.js  
│       │   ├── index.js  
│       │   ├── App.css  
│       │   └── components/  
│       └──  
└── package.json  
    └── README.md
```

■ Explanation of Each Folder & File

1. node_modules/

- Contains all installed dependencies (libraries)
- Automatically created when you run `npm install`

■ Example:

- React
- ReactDOM
- Axios

■ Important Point:

- Do NOT modify manually

2. public/ Folder

This folder contains static files.

■ index.html

- The main HTML file
- Contains a `

■ Example:

```
<body>  
  <div id="root"></div>  
</body>
```

■ This is where the React app is rendered

3. src/ Folder (Most Important)

This is the main working directory where all React code is written.

■ index.js (Entry Point)

- The starting point of the React application

- Renders the main component (`App`) into the DOM

■ Example:

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<App />);
```

■ Flow:

- Loads App component → renders into `index.html`

■ App.js (Main Component)

- Root component of the application
- Acts as a container for other components

■ Example:

```
function App() {
  return (
    <div>
      <h1>Hello React</h1>
    </div>
  );
}

export default App;
```

■ App.css

- Used for styling the App component

■ Example:

```
h1 {
  color: blue;
}
```

■ components/ Folder

- Used to store reusable components

■ Example:

```
components/
  Navbar.js
  Footer.js
```

■ Example Component:

```
function Navbar() {  
  return <h2>This is Navbar</h2>;  
}  
  
export default Navbar;
```

■ Working Flow of React Application

Step-by-step flow:

1. Browser loads index.html
2. React loads index.js
3. `index.js` renders App.js
4. App loads other components
5. UI is displayed in browser

■ Flow Diagram (Important for Exams):

```
index.html → index.js → App.js → Components → UI
```

■ Component-Based Architecture

React divides UI into small parts:

Example:

- App
- Navbar
- Content
- Footer

■ Benefit:

- Reusability
- Easy debugging
- Clean structure

■ Real-World Example

In an e-commerce website:

- App → Main page
- Navbar → Navigation menu
- Product → Product card
- Cart → Shopping cart

Each part is a separate component

■ Advantages of React Structure

- Organized code

- Easy to maintain
- Scalable for large applications
- Reusable components
- Separation of concerns

■ Disadvantages

- Initial setup may be confusing
- Too many files for beginners
- Requires understanding of folder structure

■ Conclusion

The simple React structure provides a well-organized way to build applications using components and modular files. It improves maintainability, scalability, and readability, making it ideal for modern web development.

TIP: Exam Writing Tips

- Draw folder structure diagram (very important)
- Explain:
 - `index.js` (entry point)
 - `App.js` (root component)
 - `public/index.html`
- Include flow: index.html → index.js → App → Components
- Add one small code example

Virtual DOM in React (Very Important)

■ Definition

The Virtual DOM (Document Object Model) is a lightweight copy of the real DOM, maintained in memory by React, which is used to improve performance by updating only the necessary parts of the UI instead of re-rendering the entire page.

■ Introduction

In traditional web development, whenever a change occurs (like clicking a button), the real DOM is updated directly. However, updating the real DOM is slow and inefficient, especially for large applications.

React solves this problem using the Virtual DOM, which acts as an intermediate layer between the UI and the real DOM. It ensures that only the required changes are applied, making applications faster and more efficient.

■ What is DOM?

- DOM stands for Document Object Model
- It represents the structure of an HTML document as a tree

■ Example:

```
<div>
  <h1>Hello</h1>
</div>
```

■ DOM Tree:

```
div
  ■■■ h1
    ■■■ Hello
```

■ Problems with Real DOM

1. Slow Updates
 - Every change updates the entire DOM
 - Expensive operations
2. Inefficient Rendering
 - Even small changes cause full re-render
3. Performance Issues
 - Large applications become slow

■ What is Virtual DOM?

The Virtual DOM is:

- A JavaScript object representation of the real DOM
- Stored in memory
- Faster to manipulate than real DOM

■ React uses it to compare changes and update only necessary elements.

■ Working of Virtual DOM (Step-by-Step)

This is the most important part for exams.

■ Step 1: Initial Rendering

- React creates a Virtual DOM from JSX
- It is rendered to the real DOM

■ Step 2: State Change Occurs

Example:

```
setCount(count + 1);
```

- React creates a new Virtual DOM

■ Step 3: Diffing Process

- React compares:

- Old Virtual DOM
- New Virtual DOM

■ This process is called Diffing

■ Step 4: Reconciliation

- React finds the differences
- Updates only changed elements in real DOM

■ This process is called Reconciliation

■ Step 5: Efficient Update

- Only necessary parts are updated
- Improves performance

■ Diagram Explanation (Write in Exam)

JSX → Virtual DOM → Diffing → Reconciliation → Real DOM Update

■ Example

```
import React, { useState } from "react";

function Example() {
  const [text, setText] = useState("Hello");

  return (
    <div>
      <h1>{text}</h1>
      <button onClick={() => setText("Hi")}>
        Change Text
      </button>
    </div>
  );
}
```

■ What happens:

- Only `

` text changes
- Entire page is NOT reloaded

■ Real DOM vs Virtual DOM (Important Table)

Feature	Real DOM	Virtual DOM
-----	-----	-----
Update Speed	Slow	Fast
Update Type	Full update	Partial update

Feature	Real DOM	Virtual DOM
Efficiency	Low	High
Manipulation	Direct	Indirect (via React)
Performance	Poor for large apps	Optimized

■ Advantages of Virtual DOM

- Faster UI updates
- Better performance
- Reduces unnecessary DOM operations
- Improves user experience
- Efficient rendering

■ Disadvantages

- Extra memory usage (stores copy)
- Requires understanding of React internals
- Not always fastest for very simple apps

■ Conclusion

The Virtual DOM is one of the core features of React, enabling fast and efficient updates by minimizing direct interaction with the real DOM. It plays a crucial role in making React applications highly performant and scalable.

TIP: Exam Writing Tips

- Always include:
- Definition
- Working steps (Diffing + Reconciliation)
- Draw diagram:

Virtual DOM → Compare → Update Real DOM

- Mention keywords:
- Diffing*
- Reconciliation*
- Efficient updates*
- Include one example

React Components (Core Concept – Very Important)

■ Definition

A React Component is a reusable and independent piece of UI that represents a part of the user interface. Components are the building blocks of a React application.

■ Introduction

In traditional web development, the UI is written as a single large file, which becomes difficult to manage. React solves this by breaking the UI into small, reusable components.

Each component:

- Has its own logic
- Can be reused multiple times
- Can be combined to build complex UIs

■ Example:

A webpage can be divided into:

- Header
- Navbar
- Content
- Footer

Each of these can be a separate component.

■ Characteristics of React Components

- Reusable
- Independent
- Maintainable
- Composable (can be combined)

■ Types of React Components

React components are mainly of two types:

1. Functional Components

■ Definition

Functional components are JavaScript functions that return JSX (UI).

■ Features

- Simple and easy to write
- Use Hooks for state and lifecycle
- Preferred in modern React

■ Example

```
function Welcome() {  
  return <h1>Hello Student</h1>;  
}  
  
export default Welcome;
```

■ With Props Example


```
function Welcome(props) {  
  return <h1>Hello {props.name}</h1>;  
}
```

Usage:

```
<Welcome name="John" />
```

2. Class Components

■ Definition

Class components are ES6 classes that extend `React.Component` and use a `render()` method to return JSX.

■ Features

- Use lifecycle methods
- Manage state using `this.state`
- Older approach (less used now)

■ Example

```
import React, { Component } from "react";  
  
class Welcome extends Component {  
  render() {  
    return <h1>Hello Student</h1>;  
  }  
}  
  
export default Welcome;
```

■ With State Example

```
class Counter extends React.Component {  
  constructor() {  
    super();  
    this.state = { count: 0 };  
  }  
  
  render() {  
    return (  
      <button onClick={() => this.setState({ count: this.state.count + 1 })}>  
        Count: {this.state.count}  
      </button>  
    );  
  }  
}
```

■ Functional vs Class Components (Important)

Feature	Functional Component	Class Component
-----	-----	-----
Syntax	Simple function	ES6 class
State	useState Hook	this.state
Lifecycle Methods	useEffect Hook	Built-in lifecycle methods
Usage	Modern React	Older React
Complexity	Less	More

■ Component Composition

React allows combining components:

■ Example:

```
function App() {
  return (
    <div>
      <Header />
      <Content />
      <Footer />
    </div>
  );
}
```

■ This is called component composition

■ Real-World Example

In a social media app:

- Profile → Component
- Post → Component
- Like Button → Component
- Comment Section → Component

■ Each part works independently

■ Advantages of Components

- Reusability
- Easy maintenance
- Better organization
- Faster development
- Scalability

■ Disadvantages

- Too many components can increase complexity
- Requires proper structuring
- Learning curve for beginners

■ Conclusion

React components are the foundation of React applications, enabling developers to build complex UIs by breaking them into smaller, reusable, and manageable pieces. Understanding components is essential for mastering React development.

TIP: Exam Writing Tips

- Start with definition + importance
- Clearly explain:
 - Functional components
 - Class components
- Include at least one code example for each
- Add comparison table (very important)
- Mention component composition

Introducing React Components (JSX & Rendering Basics)

■ Definition

- Introducing React Components refers to understanding how components are defined, written using JSX, and rendered in a React application to display UI elements dynamically.

■ Introduction

After understanding what components are, the next step is to learn how React introduces and uses components in real applications.

React components are not written using plain HTML. Instead, React uses a special syntax called JSX (JavaScript XML), which allows developers to write HTML-like code inside JavaScript.

React then renders these components into the browser using a process called rendering.

■ What is JSX?

■ Definition

- JSX (JavaScript XML) is a syntax extension for JavaScript that allows writing HTML-like structure inside JavaScript code.

■ Example of JSX

```
function App() {  
  return <h1>Hello React</h1>;  
}
```

■ This looks like HTML but is actually JavaScript.

■ Without JSX (Pure JavaScript)

```
function App() {  
  return React.createElement("h1", null, "Hello React");  
}
```

■ JSX makes code:

- Cleaner
- Easier to read
- Less complex

■ Rules of JSX (Important for Exams)

1. Must return a single parent element
2. Tags must be properly closed
3. Use `className` instead of `class`
4. JavaScript expressions use `{}`

■ Example with Expressions

```
function App() {  
  const name = "John";  
  return <h1>Hello {name}</h1>;  
}
```

■ Rendering in React

■ Definition

- Rendering is the process of displaying React components on the browser (UI).

■ How Rendering Works

React uses `ReactDOM` to render components into the DOM.

■ Example

```
import React from "react";  
import ReactDOM from "react-dom/client";  
import App from "./App";  
  
const root = ReactDOM.createRoot(document.getElementById("root"));  
root.render(<App />);
```

■ Explanation:

- `createRoot()` → Creates root container
- `render()` → Displays component

■ Types of Rendering

1. Initial Rendering

- First time component is displayed

2. Re-rendering

- Happens when:
- State changes
- Props change

■ React updates only necessary parts (using Virtual DOM)

■ Rendering Multiple Components

```
function Header() {  
  return <h1>Header</h1>;  
}  
  
function App() {  
  return (  
    <div>  
      <Header />  
      <p>This is content</p>  
    </div>  
  );  
}
```

■ Multiple components can be rendered together

■ Embedding Components Inside Components

```
function Button() {  
  return <button>Click Me</button>;  
}  
  
function App() {  
  return (  
    <div>  
      <Button />  
      <Button />  
    </div>  
  );  
}
```

■ Reusability of components

■ Real-World Example

In a blog website:

- App → Main page
- Post → Component
- Comment → Component

Each component is written using JSX and rendered dynamically.

■ Advantages of JSX & Rendering

- Easy to write UI
- Improves readability
- Allows dynamic content
- Efficient updates using Virtual DOM

■ Disadvantages

- Requires learning new syntax (JSX)
- Mixing HTML with JS may confuse beginners
- Needs compilation (Babel)

■ Conclusion

Introducing React components involves understanding JSX and rendering, which are essential for displaying UI in React. JSX simplifies UI creation, while rendering ensures efficient updates and display of components.

TIP: Exam Writing Tips

- Define both:

* JSX

- Rendering
- Include:
 - One JSX example
 - One rendering example
 - Mention JSX rules (important)
 - Explain initial rendering + re-rendering
- Use keywords:
 - ReactDOM*
 - render()*
 - Virtual DOM*

Creating Components in React (Steps & Best Practices)

■ Definition

- Creating Components in React refers to the process of designing, writing, and organizing reusable UI elements using functional or class-based approaches to build a structured React application.

■ Introduction

Components are the core building blocks of React. After understanding what components are and how they work, the next step is learning how to create and organize them properly.

Creating components allows developers to:

- Divide UI into smaller parts
- Reuse code efficiently

- Maintain clean and scalable applications

■ Steps to Create a Component in React

■ Step 1: Create a Component File

- Inside the `src/` folder, create a new file

■ Example:

```
src/components/Header.js
```

■ Step 2: Define the Component

You can create components in two ways:

Functional Component (Recommended)

```
function Header() {  
  return <h1>This is Header Component</h1>;  
}  
  
export default Header;
```

Class Component

```
import React, { Component } from "react";  
  
class Header extends Component {  
  render() {  
    return <h1>This is Header Component</h1>;  
  }  
}  
  
export default Header;
```

■ Step 3: Export the Component

- Use `export default` to make the component available

■ Important:

Without exporting, the component cannot be used in other files.

■ Step 4: Import the Component

- Import the component into another file (like `App.js`)

```
import Header from "../components/Header";
```

■ Step 5: Use the Component

```
function App() {  
  return (  
    <div>  
      <Header />  
    </div>  
  );  
}
```

■ Now the component will be displayed in UI

■ Using Props in Components

Props allow passing data from one component to another.

■ Example

```
function Welcome(props) {  
  return <h1>Hello {props.name}</h1>;  
}
```

Usage:

```
<Welcome name="John" />
```

■ Output:

```
Hello John
```

■ Component Reusability

One component can be used multiple times:

```
<Welcome name="Mike" />  
<Welcome name="John" />
```

■ This reduces code duplication

■ Component Composition

Combining multiple components:


```
function App() {  
  return (  
    <div>  
      <Header />  
      <Content />  
      <Footer />  
    </div>  
  );  
}
```

■ Builds complex UI from simple components

■ Best Practices for Creating Components

1. Use Functional Components
 - Simpler and modern approach
2. Keep Components Small
 - One component → one responsibility
3. Use Meaningful Names
 - Example: `Navbar`, `LoginForm`
4. Organize in Folders

```
components/  
  Header.js  
  Footer.js
```

5. Reuse Components
 - Avoid duplicate code
6. Use Props for Data Passing
 - Makes components dynamic

■ Real-World Example

In a food delivery app:

- RestaurantCard → Component
- MenuItem → Component
- Cart → Component

■ Each component is created separately and reused

■ Advantages of Creating Components

- Code reusability
- Easy maintenance
- Better readability
- Faster development
- Scalable applications

■ Disadvantages

- Too many components can be confusing
- Requires proper planning
- Beginners may find structure complex

■ Conclusion

Creating components is a fundamental skill in React development. By following proper steps and best practices, developers can build modular, reusable, and maintainable applications efficiently.

TIP: Exam Writing Tips

- Write step-by-step process (very important)
- Include:
 - Create → Export → Import → Use
 - Add one props example
 - Mention best practices
 - Use keywords:
 - Reusable*
 - Component-based*
 - Modular structure*

Data and Data Flow in React (Props, State & One-Way Data Binding)

■ Very Important

■ Definition

- Data and Data Flow in React refers to how data is stored, managed, and transferred between components using props and state, following a one-way (unidirectional) data flow.

■ Introduction

In any web application, data plays a crucial role. React provides a structured way to manage data using:

- Props (Properties) → for passing data
- State → for managing internal data
- One-way data flow → for predictable behavior

■ This structured approach solves problems like data inconsistency and synchronization issues.

1. Props (Properties)

■ Definition

- Props are used to pass data from a parent component to a child component.

■ Key Characteristics

- Read-only (immutable)
- Passed from parent → child
- Cannot be modified by child

■ Example

Parent Component:

```
function App() {  
  return <Welcome name="John" />;  
}
```

Child Component:

```
function Welcome(props) {  
  return <h1>Hello {props.name}</h1>;  
}
```

■ Output:

```
Hello John
```

■ Explanation

- `App` sends data (`name="John"`)
- `Welcome` receives it as `props.name`

■ This is data passing

2. State

■ Definition

- State is a built-in object used to store and manage dynamic data inside a component.

■ Key Characteristics

- Mutable (can change)
- Controlled within component
- Causes re-render when updated

■ Example (Functional Component)

```
import React, { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>{count}</h1>
      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </div>
  );
}
```

■ Explanation

- `useState(0)` → initial value
- `count` → current state
- `setCount()` → updates state

■ When state changes → UI updates automatically

3. Props vs State (Very Important Table)

Feature	Props	State
-----	-----	-----
Definition	Data passed from parent	Internal component data
Mutability	Immutable (read-only)	Mutable
Ownership	Parent component	Own component
Usage	Communication	Dynamic behavior
Update	Cannot be changed directly	Can be updated using setter

4. One-Way Data Binding (Unidirectional Flow)

■ Definition

- One-way data binding means that data flows in only one direction, from parent to child components.

■ Flow Diagram (Important for Exams)

Parent Component → Child Component → UI

■ Example

```
function Child(props) {
  return <h2>{props.message}</h2>;
}

function Parent() {
  return <Child message="Hello Student" />;
}
```

■ Data flows:

- Parent → Child
- Not Child → Parent

■ Why One-Way Data Flow?

- Makes application predictable
- Easy debugging
- Avoids data conflicts

5. Data Flow with State + Props

■ Combined Example

```
import React, { useState } from "react";

function Child(props) {
  return <h2>Count: {props.count}</h2>;
}

function Parent() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <Child count={count} />
      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </div>
  );
}
```

■ Explanation

- State is stored in Parent
- Passed as props to Child
- When state updates → Child re-renders

6. Real-World Example

In a shopping cart application:

- Parent → Cart component
- Child → Product component

■ Flow:

- Cart holds state (items count)
- Passes data to Product
- Product displays updated value

■ Advantages of React Data Flow

- Predictable behavior
- Easy debugging
- Better data management
- Avoids synchronization issues
- Clean architecture

■ Disadvantages

- Difficult for deeply nested components (prop drilling)
- Requires additional tools (Redux, Context API) for large apps

■ Conclusion

Data and data flow in React are managed using props and state, following a one-way data binding approach. This ensures that applications remain predictable, efficient, and easy to maintain, making React highly suitable for complex UI development.

TIP: Exam Writing Tips

- Must include:
- Props definition + example
- State definition + example
- Draw flow:

Parent → Child

- Add Props vs State table (very important)
- Mention:
- One-way data binding*
- Re-rendering*
- Include one combined example

Rendering in React (Conditional Rendering, Lists & Keys) ■ Important

■ Definition

- Rendering in React is the process of displaying components and updating the UI in the browser based on state and props changes. React ensures efficient rendering using the Virtual DOM.

■ Introduction

Rendering is a core concept in React that determines what should be shown on the screen and when. React follows a declarative approach, where developers describe the UI, and React handles the updates

automatically.

Whenever:

- State changes
- Props change

■ React re-renders the component and updates only necessary parts.

■ Types of Rendering in React

1. Initial Rendering

- When the component is displayed for the first time

■ Example:

```
function App() {  
  return <h1>Welcome</h1>;  
}
```

2. Re-rendering

- Happens when:
- State changes
- Props change

■ React updates only changed parts using Virtual DOM

■ Conditional Rendering

■ Definition

- Conditional Rendering means displaying UI elements based on a condition (like if-else).

■ Methods of Conditional Rendering

1. Using if-else

```
function App() {  
  const isLoggedIn = true;  
  
  if (isLoggedIn) {  
    return <h1>Welcome User</h1>;  
  } else {  
    return <h1>Please Login</h1>;  
  }  
}
```

2. Using Ternary Operator

```
function App() {
  const isLoggedIn = false;

  return (
    <h1>
      {isLoggedIn ? "Welcome User" : "Please Login"}
    </h1>
  );
}
```

3. Using Logical AND (&&)

```
function App() {
  const show = true;

  return (
    <div>
      {show && <h1>Hello Student</h1>}
    </div>
  );
}
```

■ Explanation

- React checks condition
- Displays corresponding UI
- Updates automatically when condition changes

■ Rendering Lists in React

■ Definition

Rendering lists means displaying multiple elements dynamically using arrays.

■ Example

```
function App() {
  const fruits = ["Apple", "Banana", "Mango"];

  return (
    <ul>
      {fruits.map((fruit) => (
        <li>{fruit}</li>
      ))}
    </ul>
  );
}
```

■ Explanation

- `map()` is used to iterate

- Each item is rendered dynamically

■ Keys in React (Very Important)

■ Definition

A key is a special attribute used to identify elements uniquely in a list.

■ Why Keys are Needed?

- Helps React identify which items changed
- Improves performance
- Avoids unnecessary re-rendering

■ Example with Keys

```
function App() {  
  const fruits = ["Apple", "Banana", "Mango"];  
  
  return (  
    <ul>  
      {fruits.map((fruit, index) => (  
        <li key={index}>{fruit}</li>  
      ))}  
    </ul>  
  );  
}
```

■ Best Practice

- Use unique IDs instead of index

```
<li key={fruit.id}>{fruit.name}</li>
```

■ Real-World Example

In an e-commerce website:

- Product list is rendered using arrays
- Conditional rendering shows:
 - “Out of Stock”
 - “Add to Cart”

■ React updates UI dynamically

■ Advantages of React Rendering

- Efficient updates using Virtual DOM
- Dynamic UI changes
- Easy conditional display
- Faster performance

■ Disadvantages

- Keys misuse can cause bugs
- Beginners may confuse conditions
- Large lists may need optimization

■ Conclusion

Rendering in React allows dynamic and efficient UI updates. With features like conditional rendering, list rendering, and keys, React ensures better performance and user experience.

TIP: Exam Writing Tips

- Define Rendering clearly
- Explain:
 - Conditional rendering (3 methods)
 - List rendering
 - Keys (very important)
- Include:
 - At least 2 code examples
- Mention:
 - Virtual DOM*
 - Efficient updates*
- Highlight:
 - Keys improve performance

Life Cycle Methods in React (Mounting, Updating, Unmounting & Hooks) ■ Very Important

■ Definition

- Lifecycle Methods in React are special methods that are automatically called at different stages of a component's life, such as when it is created, updated, or removed from the DOM.

■ Introduction

Every React component goes through a lifecycle, similar to a human life cycle:

- Born → Mounted
- Lives → Updates
- Dies → Unmounted

These stages help developers:

- Control component behavior
- Perform operations like API calls, updates, cleanup

■ Lifecycle methods are mainly used in class components, while Hooks are used in functional components.

■ Phases of Lifecycle (Very Important)

React lifecycle has 3 main phases:

1. Mounting Phase

2. Updating Phase
3. Unmounting Phase
1. Mounting Phase

■ Definition

Mounting is the phase when a component is created and inserted into the DOM.

■ Important Methods

■ constructor()

- Called first
- Used to initialize state

```
constructor() {  
  super();  
  this.state = { count: 0 };  
}
```

■ render()

- Returns JSX (UI)
- Required method

```
render() {  
  return <h1>Hello</h1>;  
}
```

■ componentDidMount()

- Called after component is added to DOM
- Used for:
 - API calls
 - Data fetching

```
componentDidMount() {  
  console.log("Component Mounted");  
}
```

2. Updating Phase

■ Definition

Updating occurs when:

- State changes
- Props change

■ Important Methods

■ **shouldComponentUpdate()**

- Decides whether to re-render or not

■ **render()**

- Updates UI

■ **componentDidUpdate()**

- Called after update
- Used for:
 - API calls
 - Side effects

```
componentDidUpdate() {  
  console.log("Component Updated");  
}
```

3. Unmounting Phase

■ **Definition**

Unmounting is when a component is removed from the DOM.

■ **Important Method**

■ **componentWillUnmount()**

- Called before component is destroyed
- Used for:
 - Cleanup
 - Removing timers
 - Cancelling API calls

```
componentWillUnmount() {  
  console.log("Component Removed");  
}
```

■ **Lifecycle Flow Diagram (Write in Exam)**

Mounting → Updating → Unmounting

■ **Lifecycle in Functional Components (Hooks)**

Modern React uses Hooks instead of class lifecycle methods.

■ **useEffect Hook**

■ **Definition**

`useEffect` is used to perform side effects like:

- API calls
- DOM updates
- Subscriptions

■ Example

```
import React, { useEffect } from "react";

function App() {
  useEffect(() => {
    console.log("Component Mounted");

    return () => {
      console.log("Component Unmounted");
    };
  }, []);

  return <h1>Hello React</h1>;
}
```

■ Explanation

- Runs after rendering
- `[]` → runs only once (mounting)
- Return function → cleanup (unmounting)

■ Class vs Functional Lifecycle (Important)

Feature	Class Component	Functional Component
-----	-----	-----
Lifecycle Handling	Methods	Hooks (useEffect)
Complexity	More	Less
Modern Usage	Less used	Preferred

■ Real-World Example

In a weather app:

- Mounting → Fetch weather data
- Updating → Refresh data on change
- Unmounting → Stop API calls

■ Advantages of Lifecycle Methods

- Control component behavior
- Handle side effects
- Manage resources efficiently
- Improve performance

■ Disadvantages

- Complex in class components
- Hard to manage multiple methods
- Requires understanding of phases

■ Conclusion

Lifecycle methods help manage different stages of a component's life. While class components use lifecycle methods, modern React uses Hooks like `useEffect`, making code simpler and more efficient.

TIP: Exam Writing Tips

- Clearly explain 3 phases:
 - Mounting
 - Updating
 - Unmounting
- Mention key methods:
 - `componentDidMount`
 - `componentDidUpdate`
 - `componentWillUnmount`
- Include `useEffect` example (very important)
- Draw lifecycle flow diagram
- Use keywords:
 - Side effects*
 - Component lifecycle*
 - Hooks*

Working with Forms in React (Controlled Components & Handling Input) ■ Important

■ Definition

- Working with Forms in React refers to handling user input (like text fields, checkboxes, buttons) using React's state, where form elements are controlled by the component.

■ Introduction

Forms are essential in web applications for:

- User login
- Registration
- Data submission

In traditional HTML, form data is handled using the DOM.

But in React, forms are handled using state, making them more dynamic and controlled.

■ React mainly uses Controlled Components for handling forms.

■ Controlled Components

■ Definition

A Controlled Component is a form element whose value is controlled by React state.

■ React becomes the single source of truth for form data.

■ Example (Text Input)

```
import React, { useState } from "react";

function FormExample() {
  const [name, setName] = useState("");

  return (
    <div>
      <input
        type="text"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />
      <h2>Your Name: {name}</h2>
    </div>
  );
}
```

■ Explanation

- `value={name}` → controlled by state
- `onChange` → updates state
- Input and state are synchronized

■ This ensures:

- Real-time updates
- Better control over input

■ Handling Multiple Inputs

■ Example

```
import React, { useState } from "react";

function Form() {
  const [formData, setFormData] = useState({
    name: "",
    email: ""
  });

  const handleChange = (e) => {
    setFormData({
      ...formData,
      [e.target.name]: e.target.value
    });
  };

  return (
    <div>
      <input name="name" onChange={handleChange} />
      <input name="email" onChange={handleChange} />
    </div>
  );
}
```

■ Explanation

- Uses single state object
- Updates specific field dynamically
- Efficient for large forms

■ Form Submission

■ Example

```
function FormSubmit() {
  const [name, setName] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    alert("Submitted: " + name);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input onChange={(e) => setName(e.target.value)} />
      <button type="submit">Submit</button>
    </form>
  );
}
```

■ Explanation

- `e.preventDefault()` → prevents page reload
- Handles form data in React

■ Uncontrolled Components

■ Definition

Uncontrolled components store data in the DOM instead of state.

■ Example

```
import React, { useRef } from "react";

function Uncontrolled() {
  const inputRef = useRef();

  const handleSubmit = () => {
    alert(inputRef.current.value);
  };

  return (
    <div>
      <input ref={inputRef} />
      <button onClick={handleSubmit}>Submit</button>
    </div>
  );
}
```

■ Explanation

- Uses `useRef`
- Data is accessed directly from DOM

■ Controlled vs Uncontrolled Components

Feature	Controlled Component	Uncontrolled Component
-----	-----	-----
Data Storage	React State	DOM
Control	Full control	Less control
Validation	Easy	Difficult
Usage	Preferred	Rarely used

■ Form Validation (Basic)

■ Example

```
function Validation() {
  const [email, setEmail] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();

    if (!email.includes("@")) {
      alert("Invalid Email");
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input onChange={(e) => setEmail(e.target.value)} />
      <button>Submit</button>
    </form>
  );
}
```

■ Real-World Example

In a login form:

- Username → Controlled input
- Password → Controlled input
- Submit → Validates and sends data

■ React handles everything using state

■ Advantages of React Forms

- Full control over input
- Real-time validation
- Easy data handling
- Better user experience

■ Disadvantages

- More code compared to HTML forms
- Beginners may find it complex
- Requires state management

■ Conclusion

React forms use controlled components to manage user input efficiently. By using state, React ensures better control, validation, and synchronization of data, making forms more powerful and dynamic.

TIP: Exam Writing Tips

- Clearly define:
- Controlled components
- Uncontrolled components
- Include:
- One input example
- One form submission example

- Add comparison table
- Mention:
- useState*
- onChange*
- preventDefault()*
- Highlight:
- Controlled components are preferred

Integrating Third-Party Libraries in React (Axios, Bootstrap, etc.) ■ Important

■ Definition

- Integrating Third-Party Libraries in React refers to the process of adding external packages or tools (like Axios, Bootstrap, etc.) into a React application to extend functionality and improve development efficiency.

■ Introduction

React itself is a UI library, not a complete framework. It does not provide built-in features like:

- HTTP requests
- Styling systems
- Advanced utilities

To overcome this, developers use third-party libraries.

■ These libraries help:

- Reduce development time
- Avoid writing complex code
- Add advanced features easily

■ Why We Need Third-Party Libraries

1. To Perform API Calls
 - React does not have built-in HTTP support
 - Use libraries like Axios
2. For Styling
 - CSS can be complex
 - Use libraries like Bootstrap
3. For Routing
 - React does not include routing
 - Use React Router
4. For State Management
 - For large apps
 - Use Redux, Context API

■ Installing Third-Party Libraries

■ Using npm (Node Package Manager)

```
npm install axios
npm install bootstrap
```

■ Using yarn

```
yarn add axios
yarn add bootstrap
```

■ After installation, libraries are added to:

```
node_modules/
```

■ Example 1: Using Axios (API Calls)

■ What is Axios?

Axios is a library used to make HTTP requests (GET, POST, etc.)

■ Example

```
import React, { useEffect, useState } from "react";
import axios from "axios";

function Users() {
  const [data, setData] = useState([]);

  useEffect(() => {
    axios.get("https://jsonplaceholder.typicode.com/users")
      .then((response) => {
        setData(response.data);
      });
  }, []);

  return (
    <ul>
      {data.map((user) => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

■ Explanation

- `axios.get()` → fetch data
- `useEffect()` → runs on component mount

- Data stored in state
- Rendered using `map()`

■ Example 2: Using Bootstrap (Styling)

■ Installation

```
npm install bootstrap
```

■ Import Bootstrap

```
import "bootstrap/dist/css/bootstrap.min.css";
```

■ Example

```
function App() {
  return (
    <div className="container">
      <button className="btn btn-primary">
        Click Me
      </button>
    </div>
  );
}
```

■ Explanation

- `container` → layout
- `btn btn-primary` → styled button

■ No need to write custom CSS

■ Commonly Used Third-Party Libraries

Library	Purpose
-----	-----
Axios	API requests
Bootstrap	Styling
React Router	Navigation
Redux	State management
Material UI	UI components

■ Real-World Example

In an e-commerce app:

- Axios → Fetch products from API
- Bootstrap → Style product cards
- React Router → Navigate pages

■ All libraries work together

■ Advantages of Third-Party Libraries

- Saves development time
- Provides advanced features
- Reduces code complexity
- Improves UI and functionality
- Widely tested and reliable

■ Disadvantages

- Increases project size
- Dependency on external packages
- Version compatibility issues
- Learning curve

■ Best Practices

- Install only necessary libraries
- Keep libraries updated
- Avoid too many dependencies
- Check documentation before use

■ Conclusion

Third-party libraries play a crucial role in React development by adding powerful features like API handling, styling, and routing. They make development faster, easier, and more efficient.

TIP: Exam Writing Tips

- Define third-party libraries clearly
- Include:
 - Axios example (very important)
 - Bootstrap example
 - Mention installation commands (`npm install`)
 - Add table of libraries
- Use keywords:
 - API calls*
 - Styling*
 - External packages*

Routing in React (React Router – Navigation Between Pages) ■ Very Important

■ Definition

- Routing in React is the process of navigating between different components/pages in a single-page application (SPA) without reloading the entire page.

■ Introduction

In traditional websites:

- Each page load → full reload

In React:

- Navigation happens without refreshing the page
- Only required components are updated

■ This is achieved using a library called React Router

■ Why Routing is Needed

- To navigate between pages (Home, About, Contact)
- To create SPA (Single Page Application)
- To improve performance and user experience

■ React Router Basics

■ Installation

```
npm install react-router-dom
```

■ Key Components

- `BrowserRouter` → Wraps entire app
- `Routes` → Defines route container
- `Route` → Defines path + component
- `Link` → Navigation without reload

Method 1: Using `createBrowserRouter` + Array of Objects

■ Explanation

In this method:

- Routes are defined as an array of objects
- Each object represents a route
- Used with `RouterProvider`

■ Example

```
import React from "react";
import {
  createBrowserRouter,
  RouterProvider
} from "react-router-dom";

// Components
function Home() {
  return <h1>Home Page</h1>;
}

function About() {
  return <h1>About Page</h1>;
}

// Route Configuration (Array of Objects)
const router = createBrowserRouter([
  {
    path: "/",
    element: <Home />
  },
  {
    path: "/about",
    element: <About />
  }
]);

function App() {
  return <RouterProvider router={router} />;
}

export default App;
```

■ Explanation

- `createBrowserRouter()` → Creates router
- Routes defined as array of objects
- Each object contains:
 - `path` → URL
 - `element` → Component

■ This is the modern recommended approach

Method 2: Using `BrowserRouter` + `<Route>` Tags

■ Explanation

In this method:

- Routes are defined using `<Route>` components
- Wrapped inside `<BrowserRouter>`

■ Example

```
import React from "react";
import {
  BrowserRouter,
  Routes,
  Route,
  Link
} from "react-router-dom";

// Components
function Home() {
  return <h1>Home Page</h1>;
}

function About() {
  return <h1>About Page</h1>;
}

function App() {
  return (
    <BrowserRouter>

      {/* Navigation */}
      <nav>
        <Link to="/">Home</Link> |
        <Link to="/about">About</Link>
      </nav>

      {/* Routes */}
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>

    </BrowserRouter>
  );
}

export default App;
```

■ Explanation

- `BrowserRouter` → Wraps app
- `Routes` → Holds all routes
- `Route` → Defines path + component
- `Link` → Used for navigation

■ Difference Between Two Methods (Important)

Feature	createBrowserRouter (Modern)	BrowserRouter + Route (Traditional)
-----	-----	-----

Feature	createBrowserRouter (Modern)	BrowserRouter + Route (Traditional)
Syntax	Array of objects	JSX-based
Flexibility	High	Medium
Usage	Modern React Router v6+	Older / common
Structure	Centralized	Distributed

■ Navigation Example (Important)

```
import { Link } from "react-router-dom";

<Link to="/about">Go to About</Link>
```

■ Works without page reload

■ Real-World Example

In an e-commerce app:

- `/` → Home page
- `/products` → Product list
- `/cart` → Shopping cart

■ Routing allows smooth navigation

■ Advantages of React Routing

- No page reload
- Faster navigation
- Better user experience
- Supports SPA architecture

■ Disadvantages

- Requires setup
- Learning curve
- Extra dependency (react-router-dom)

■ Conclusion

Routing in React enables smooth navigation between pages using React Router. Both methods—createBrowserRouter (modern) and BrowserRouter with Route (traditional)—allow developers to build efficient single-page applications.

TIP: Exam Writing Tips

- Define routing clearly
- Mention:
 - SPA concept
 - React Router

- MUST include:
- Two methods (very important question)
- Add:
- One example for each method
- Mention keywords:
- Navigation*
- BrowserRouter*
- createBrowserRouter*
- Highlight:
- Modern vs Traditional approach

UNIT-V COMPLETED
